

Trabajo Práctico Lenguajes Formales y Autómatas

“Motor de búsqueda de funciones similares vía AST”

AÑO: 2026

PROFESORES: Mariano Ferrero, Ariel Rossanigo

N° de GRUPO: 14

INTEGRANTES: Lautaro Christiansen, Cesar Werlen, Alejandro Gareis

FECHA DE ENTREGA: 30-10-26

1. Decisiones de diseño

Para poder llevar a cabo la detección de funciones duplicadas a partir de su estructura lógica y no del texto plano, nuestro sistema se organizó en cuatro módulos independientes:
Extracción (extractor.py): Este modulo explora de forma recursiva los archivos .py del directorio indicado mediante os.walk. Con ast.parse(), procesa el código para generar su Árbol de Sintaxis Abstracta (AST) y aísla únicamente las definiciones de funciones (FunctionDef y AsyncFunctionDef), registrando para cada una su nombre, archivo de origen, línea de inicio y subárbol sintáctico.
Normalización (normalizador.py): Utiliza la herramienta ast.NodeTransformer para estandarizar el código y permitir que dos funciones con la misma lógica pero con distinta redacción resulten comparables:

a. *Nombre de la función:* Se renombra a un valor genérico (FUNC) para que el identificador original no intervenga en el cálculo.
b. *Variables y parámetros:* Se sustituyen secuencialmente por etiquetas homogéneas (VAR1, VAR2, etc.), manteniendo la coherencia de uso dentro de la función y conservando intactas las funciones nativas de Python (len, range, print).
c. *Literales:* Los números y cadenas de texto se reemplazan por la etiqueta LIT, preservando únicamente booleanos y None por su rol directo en las condiciones lógicas.

Serialización (similitud.py): Aplica un recorrido en pre-order sobre el árbol normalizado (visita primero la raíz y luego sus ramas de izquierda a derecha), transformando la estructura jerárquica en una secuencia lineal de tokens con los tipos de nodos y sus valores estandarizados.
Interfaz de búsqueda (buscar_similares.py): Gestiona la ejecución desde la terminal mediante argparse. Permite ingresar la función a buscar mediante la referencia archivo.py:nombre o introduciendo el código fuente de forma directa. Realiza la comparación contra todas las funciones del repositorio y excluye automáticamente la comparación de la función objetivo consigo misma.

2. Métrica elegida y justificación

Se utilizó la similitud de jaccard sobre trigramas (n = 3). Al transformar el AST en una lista de tokens, esta se divide en ventanas de tres elementos consecutivos contiguos, dadas dos funciones con sus conjuntos de trigramas A y B, el grado de coincidencia se calcula como:

$$J(A, B) = |(A \cap B)| / |(A \cup B)|$$

Es decir, la cantidad de trigramas compartidos dividida por el total de trigramas únicos entre ambas funciones.

Justificación de la elección:
Comparar tokens de forma individual trataría al código como una lista desordenada de palabras, perdiendo la estructura del árbol. Agrupar de a tres elementos permite capturar relaciones directas de la gramática (por ejemplo: asignación, variable y operador) sin perder la noción de jerarquía.
Elegimos la similitud de Jaccard sobre trigramas porque compara conjuntos de patrones sintácticos locales en lugar de exigir una secuencia rígida. Esto permite que, si alguien intercambia dos líneas de código independientes o agrega una sentencia intermedia, el

puntaje no se desplome drásticamente como sucedería con métricas estrictas de alineamiento como Levenshtein. Al mismo tiempo, agrupar los tokens de a tres evita perder la estructura jerárquica del AST (lo que pasaría si miráramos palabras sueltas como una simple bolsa de términos), y computacionalmente resulta ideal porque calcular la unión y la intersección de conjuntos es sumamente rápido y liviano para comparar muchas funciones contra todo un repositorio.
Fijamos un umbral de corte del 0.80 (80%) donde un Score = 1.00 representa duplicados exactos o funciones idénticas con variables y literales cambiados (Clones Tipo 1 y 2), un Score de entre 0.80 y 1 son candidatas firmes a duplicado con cambios menores, líneas invertidas o sentencias auxiliares intercaladas (Clones Tipo 3) y bajo un Score < 0.80 se descarta la copia ya que las coincidencias encontradas corresponden a construcciones habituales del lenguaje (definición de argumentos, retornos, etc.).

3. Limitaciones conocidas

El análisis sobre el AST evalúa la estructura de las reglas gramaticales y no el comportamiento del programa en ejecución, lo que presenta ciertas restricciones:
- **Clones semánticos:** No reconoce funciones que logran el mismo resultado mediante estructuras de control o algoritmos distintos. Por ejemplo, resolver un problema con un bucle while frente a una solución recursiva, o un for tradicional frente a una comprensión de listas, genera árboles sintácticos incompatibles y arroja una similitud cercana a cero a pesar de que el cómputo sea idéntico.
- **Operaciones conmutativas:** Debido a que el recorrido visita los nodos rígidamente de izquierda a derecha, expresiones lógicamente equivalentes como a + b y b + a producen secuencias de tokens distintas.

4. Posible mejora

Como posible mejora detectamos que podríamos extender la interfaz de comandos con un parámetro como --archivos (utilizando nargs="+" en argparse). Esto permitiría comparar una función contra una lista explícita de dos o más archivos puntuales elegidos por el usuario, sin obligarlo a escanear un directorio entero ni limitarlo a un archivo individual.